

20250617 日报

今日工作内容

- 1、对《误操作工具需求需要处理 tlog 及字段》需求中的 itemchange 和 logdtacntbuy 的序列化和反序列化函数进行测试。

在此过程中，学习了 golang 中对于测试函数的规定：go 中的测试函数的签名必须严格遵守规范，否则会触发编译错误。函数签名必须以 Test 为前缀，如：func TestXxx(t *testing.T)，其中 Xxx 首字母必须大写，参数必须是 *testing.T 类型，且无返回值。

所遇问题：itemchange.UnmarshalPb()反序列化代码中，结构体元素名称重复写了一个，导致测试失败。

itemchange 的序列化和反序列化测试：

```
[root@drewjjli-1t70osepea itemchangeplusredis]# go test -v -run TestElements
=== RUN    TestElements
--- PASS: TestElements (0.00s)
PASS
ok        silk_song/processor/silksongplusredis/itemchangeplusredis    0.014s
```

logdtacntbuy 的序列化和反序列化测试：

```
[root@drewjjli-1t70osepea logdtacntbuyplusredis]# go test -v -run TestElements
=== RUN    TestElements
--- PASS: TestElements (0.00s)
PASS
ok        silk_song/processor/silksongplusredis/logdtacntbuyplusredis    0.012s
```

现在已经将该需求移到下周，因此当前此需求停滞到了序列化测试完成，gdp 部署开始到了任务构建完成的阶段。

- 2、《星币转区接入新的规则排序》需求：使用在线和离线两种方式提取出需求中的几张表的 id+品质字段，并最终写入 redis。

离线处理逻辑（以《b 表情信息表》为例）：ReadFile()读取"b 表情信息表.xlsx"表->反序列化读取数据为 Sheet 结构->调用 gManager 下的 Process()(主要处理逻辑)->按照 gManager 中的获取数据 getData()->调用 gManager 的 Parse 函数，该函数用于解析 XML 数据并映射到 Item 结构体->将得到的 data 写入 redis(SaveToRedis()的调用)。其主逻辑代码如下：

```

func Process(ctx *common.Context) error {
    buf, err := os.ReadFile(path.Join(ctx.Dir, "b 表情信息表.xlsx"))
    if err != nil {
        log.ErrorContext(ctx, err)
        return err
    }
    var root common.Root
    if err := xml.Unmarshal(buf, &root); err != nil { //
[]byte->sheet
        log.ErrorContext(ctx, err)
        return err
    }
    if err := bemoijinfo.Process(ctx, &root); err != nil {
        log.ErrorContext(ctx, err)
        return err
    }
    return nil
}

```

在线处理逻辑（以《b 表情信息表》为例）：version_sync.Init()初始化，从 redis 中获取数据->main.go()注册服务到 serverImp->main_process.go 的 GetSheet()->getsheet.Process()(主要处理逻辑)->json 解析化->pb 格式(ID+品质)->整表返回。

当前需求离线部分代码基本完成，在线部分刚开始。